CS342 OPERATING SYSTEMS
Spring 2026 - Project #3
SYNCHRONIZATION and DEADLOCKS
(RSM Library)

Assigned: Mar 27, 2026
Due date: Apr 13, 2026; 23:59. Document Version: **1.2**

- The project will be done in groups of 3 students at most. You can do it individually if you wish.
- You will program in C/Linux. Programs will be tested in Ubuntu 24.04 Linux 64-bit.
- You are not allowed to post cs342 project/homework/exam assignments and/or their solutions to public places such as github. This is very important. This rule applies to graduated students as well.
- You are not allowed to look at a solution from Internet, from other sources, from previous semesters, or at the code of your friend. You should come up with your own solution and implementation. You can use your previous solutions for standard data structures like lists.

- **Objectives/keywords:** Learn and practice deadlock detection, deadlock avoidance, synchronization, semaphores, multiprocess programs, resource management, shared memory.

In this project, you will develop a **Resource Manager** (**RSM**) library that will manage a set of simulated resources shared among multiple processes. An application that is linked to this library will be able to create multiple child **processes**, each of which will be able to repeatedly request, use, and release **resources** during its lifetime. The library will use **shared memory** to maintain *shared* data structures and simulate resources.

At **startup**, the application wil indicate the number of resource types, the number of instances per type, and the number of child processes to the library. The application will then create that many child processes. The library will handle the resource requests that will be done by child processes. The child processes will run concurrently.

Resources are **simulated** resources, not physical resources. A process will be **blocked** if its request cannot be safely granted or if resources are not available. The library will also be able to **avoid deadlocks** and to **detect** deadlocks (if they are not avoided).

INTERFACE

Your library will implement the following functions.

int **rsm_init (**int **N**, int **M**, int **exist**[M], int **avoid)**:

This function initializes the shared-memory data structures required for resource management. **N** is the number of processes, and **M** is the number of resource types. The **exist** parameter is an array of size M that specifies the total number of instances for each resource type (initially all available). The **avoid** parameter indicates whether deadlock avoidance is enabled (1) or not (0). If disabled, deadlocks may occur. The function returns 0 on success and -1 on error (e.g., if N or M exceeds the maximum supported limits). This function must be called only once, by the **main process**; child **processes** should not invoke it. This function should create and initialize the **shared memory segment**.

int **rsm_destroy()**:

This function will destroy (remove) the shared memory segment and deallocate all resources. It will leave a clean environment for the next running of an application. This function will only be called by the main process; possibly after all child processes have terminated.

int **rsm_process_started** (int **apid**):

This function is called by a child process immediately after it starts execution. It informs the library that the process has started and is active. The **apid** parameter is an integer assigned by the main process as the ID of the child. Hence, it is an application-assigned PID for the child and is a unique integer in the range [0, N−1], where N is the number of child processes. This **apid** is not the system-assigned process identifier (which is of type pid_t). A process can obtain its system-assigned PID by using the getpid() system call. Internally, your library can associate the **apid** value with the system-assigned pid value for a child process.

This function must be called before any other library calls (e.g., rm_claim() or rm_request()). It must not be called by the main process. In this project, the main process is not considered a resource-requesting entity.

int **rsm_claim** (int **claim**[M]):

If deadlock avoidance is enabled (via rm_init()), a (child) process uses this function to declare its maximum resource demand. It must be called after rm_process_started() and before any resource requests. The **claim** array specifies the maximum number of instances of each resource type the process may request. The library records this information in a maximum demand matrix, which is used by the deadlock avoidance algorithm within rsm_request(). The function returns 0 on success and -1 on error (e.g., if the claim exceeds the existing instances).

int **rsm_process_ended**():

This function is called by a process just before termination to inform the library that it is exiting. It returns 0 on success and -1 on failure.

int **rsm_request (**int **request**[M]**)**:

This function is called by a process to request resources from the library. The function will allocate the requested resources if resources are available. If deadlock avoidance is used, resources may not be allocated even when they are available, because it may not be safe to do so. The **request** parameter is an integer array of size M, and indicates the number of instances of each resource type that the calling process is requesting.

If the requested resources are available and it is safe to allocate them, then they will be allocated and the function will return without getting blocked. If the requested resources are not available or it is not safe to allocate the resources, the process will be blocked inside the function, until the requested resources can be allocated. The function will return 0 upon success (resources allocated). It will return -1 if there is an error condition, for example, the number of requested instances for a resources type is greater than the number of existing instances.

int **rsm_release (**int **release**[M]**)**:

This function is called by a process to release resources. The **release** array (size M) specifies the number of instances of each resource type to free. Releasing resources may unblock other processes waiting in rm_request(). The function returns 0 on success and -1 on error (e.g., releasing more than allocated). It is possible that a process may release only part of its allocated resources; so request and release calls need not match.

int **rsm_detection()**:

This function checks for deadlocked processes. It returns the number of deadlocked processes, 0 if none, and -1 on error.

void **rsm_print_state** (char **headermsg**[]):

This function prints the current state of the library (system), including the Existing vector, Available vector, Allocation matrix, Request matrix, Maximum Demand matrix, and Need matrix. If deadlock avoidance is disabled, Maxium Demand and Need matrices contain zeros.

The **headermsg** string is printed at the start of the output. Applications can call this function anytime to inspect the library (system) state. This is the only function in the library that produces screen output.

The output format should closely match the example below, which shows two resource types (R0, R1) and three processes (P0, P1, P2) with the header "The current state".

```
##########################
The current state
###########################
Exist:
     R0 R1
     8  5

Available:
     R0 R1
     3  5

Allocation:
     R0 R1
P0:  5  0
P1:  0  0
P2:  0  0

Request:
     R0 R1
P0:  0  0
P1:  2  0
P2:  0  0

MaxDemand:
     R0 R1
P0:  8  4
P1:  8  5
P2:  0  3

Need:
     R0 R1
P0:  3  4
P1:  8  5
P2:  0  3
###########################
```

These library functions may be called concurrently by multiple processes, so your implementation must be free of **race conditions**. You should consider all **synchronization** issues and solve them. For example, a process should be blocked if it can not grab the requested resources.

The provided header file **rsm.h** defines the library interface. MAX_RT specifies the maximum number of resource types, and MAX_PR specifies the maximum number of processes that can request resources.

```
#ifndef RSM_H
#define RSM_H
```

```
#include <pthread.h>

#define MAX_RT 100 // max num of resource types supported
#define MAX_PR 100 // max num of processes supported

int rsm_init(int p_count, int r_count,
            int exist[], int avoid);
int rsm_destroy();
int rsm_process_started(int apid);
int rsm_process_ended();
int rsm_claim (int claim[]); // only for avoidance
int rsm_request (int request[]);
int rsm_release (int release[]);
int rsm_detection();
void rsm_print_state (char headermsg[]);

#endif /* RSM_H */
```

Your library will be implemented in file **rsm.c**, which will also define the global variables and structures used by the functions. Implement the functions described above; additional helper functions are allowed but should **not** be included in **rsm.h** and should not be called directly by applications.

You will use **POSIX semaphores** for synchronization. You may use as many semaphores as needed. The solution does not need to be starvation-free. You will use **POSIX shared memory**.

A starter set of files (**skeleton**) will be available on **GitHub**:
https://github.com/korpeoglu/cs342spring2026-p3.git
You can clone it to begin. The **rsm.c** skeleton is provided and must be completed.

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include "rsm.h"

int N; // number of processes
int M; // number of resource types

int ExistingV[MAX_RT];
int AvailV[MAX_RT];
int AllocationM[MAX_PR][MAX_RT];
int RequestM[MAX_PR][MAX_RT];

//  avoidance
int MaxM[MAX_PR][MAX_RT];
int NeedM[MAX_PR][MAX_RT];

#define TRUE 1
#define FALSE 0



//..... definitions/variables .....
//.....
//.....
```

```
int rsm_init(int p_count, int r_count, int exist[],  int avoid)
{
    int ret = 0;

    return  (ret);
}

int rsm_destroy()
{
    int ret = 0;

    return (ret);
}

int rsm_process_started(int pid)
{
    int ret = 0;
    return (ret);
}


int rsm_process_ended()
{
    int ret = 0;
    return (ret);
}


int rsm_claim (int claim[])
{
    int ret = 0;
    return(ret);
}

int rsm_request (int request[])
{
    int ret = 0;

    return(ret);
}


int rsm_release (int release[])
{
    int ret = 0;

    return (ret);
}


int rsm_detection()
{
    int ret = 0;

    return (ret);
}


void rsm_print_state (char hmsg[])
{
    return;
```

```
}
```

A SAMPLE APPLICATION

A multi-process application, for example **app.c**, that will use your library will first include the header file **rsm.h**. An application will be compiled and linked with your library as follows:

```
gcc -Wall -o app  -L. -lrsm  -lpthread -lrt app.c
```

Below is a sample application showing the use of the library (it may not be complete; it is not debugged).

```
#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdarg.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/wait.h>
#include "rsm.h"

#define NUMR 1        // number of resource types
#define NUMP 2        // number of processes

int AVOID = 1;
int exist[1] =  {8};  // resources existing in the system

void pr (int apid, char astr[], int m, int r[])
{
    int i;
    printf ("process %d, %s, [", apid, astr);
    for (i = 0; i < m; ++i) {
        if (i == (m-1))
            printf ("%d", r[i]);
        else
            printf ("%d,", r[i]);
    }
    printf ("]\n");
}


void setarray (int r[MAX_RT], int m, ...)
{
    va_list valist;
    int i;

    va_start (valist, m);
    for (i = 0; i < m; i++) {
        r[i] = va_arg(valist, int);
    }
    va_end(valist);
    return;
}


void func_p1 (int apid)
```

```
{
    int request1[MAX_RT];
    int request2[MAX_RT];
    int claim[MAX_RT];

    rsm_process_started (apid);

    setarray(claim, NUMR, 8);
    rsm_claim (claim);

    setarray(request1, NUMR, 5);
    pr (apid, "REQ", NUMR, request1);
    rsm_request (request1);

    sleep(4);

    setarray(request2, NUMR, 3);
    pr (apid, "REQ", NUMR, request2);
    rsm_request (request2);

    rsm_release (request1);
    rsm_release (request2);

    rsm_process_ended();
    exit(0);
}


void func_p2 (int apid)
{
    int request1[MAX_RT];
    int request2[MAX_RT];
    int claim[MAX_RT];

    rsm_process_started (apid);

    setarray(claim, NUMR, 8);
    rsm_claim (claim);

    setarray(request1, NUMR, 2);
    pr (apid, "REQ", NUMR, request1);
    rsm_request (request1);

    sleep(2);

    setarray(request2, NUMR, 4);
    pr (apid, "REQ", NUMR, request2);
    rsm_request (request2);

    rsm_release (request1);
    rsm_release (request2);

    rsm_process_ended ();
    exit(0);
}


int main(int argc, char **argv)
{
    int i;
    int count;
```

```
    int ret;
    int n;

    if (argc != 2) {
        printf ("usage: ./app avoidflag\n");
        exit (1);
    }

    AVOID = atoi (argv[1]);

    if (AVOID == 1)
        rsm_init (NUMP, NUMR, exist, 1);
    else
        rsm_init (NUMP, NUMR, exist, 0);

    i = 0;  // we select a pid for the child process
    n = fork();
    if (n == 0) {
        func_p1(i);
    }

    i = 1;  // we select a tid for the thread
    n = fork();
    if (n == 0) {
        func_p2(i);
    }

    count = 0;
    while ( count < 10) {
        sleep(1);
        rsm_print_state("The current state");
        ret = rsm_detection();
        if (ret > 0) {
            printf ("deadlock detected, count=%d\n", ret);
            rsm_print_state("state after deadlock");
        }
        count++;
    }


    for (i = 0; i < NUMP; ++i) {
            wait (NULL);
    }

    rsm_destroy();
}
```

DEVELOPING AN APPLICATION

Develop an application that will create some number of processes that will run concurrently and will request, use, and release resources concurrently. The number of processes should be at least 3, and the number of resource types should be at least 5. Your application will use your library. The library will do the allocation and release of resources. Your application should demonstrate the occurrence of deadlocks, detection of deadlocks, and avoidance of deadlocks. It is up to you how to demonstrate these, i.e., what to put into your application. You will write one application, called **myapp.c**

(executable name: **myapp**), to demonstrate deadlocks, deadlock detection, and deadlock avoidance. The application will take one command line parameter, a flag value (0 or 1). If flag is 0, deadlocks will not be avoided and your application will demonstrate the occurrence and detection of deadlocks. If flag is 1, then deadlock avoidance will be done and demonstrated.

We will also develop our test applications that will be linked with your library. Our test applications may create many processes. Each process may issue many request/release calls to the library. We will check if deadlocks occur, if they can be detected, and if they can be avoided.

As a result of compilation of the library (rsm.c), we will obtain a binary library file, called **librsm.a**. Below is a Makefile that is showing how to build the library librsm.a and how to link an application app.c with the library.

```
all: librsm.a  app

librsm.a:  rsm.c
      gcc -Wall -c rsm.c
      ar -cvq librsm.a rsm.o
      ranlib librsm.a

app: app.c
      gcc -Wall -o app app.c -L. -lrsm -lpthread

clean:
      rm -fr *.o *.a *~ a.out  app rsm.o rsm.a librsm.a
```

EXPERIMENTS and REPORT

You will design and conduct some performance experiments. You will then interpret them. You will put your results, interpretations, and conclusions in a report.pdf file. You will include this file as part of your uploaded package. Experiments and the related report will be **20% of the project grade**.

SUBMISSION

Submission will be the same as the previous project.

You should include myapp.c as well as part of your submission. Modify the Makefile so that it compiles myapp.c when we type "make". Include your Makefile in your submission.

Put all your files into a directory named with your Student ID. If the project is done as a group, the IDs of all students will be written, separated by a dash '-'. In a README.txt file, write your name, ID, etc. (if done as a group, all names and IDs will be included). The set of files in the directory will include README.txt, Makefile, report.pdf, and program source file(s). We should be able to compile your program(s) by just typing make. No binary files (executable files or .o files) will be included in your submission. Then **tar** and **gzip** the directory, and submit it to **Moodle**.

For example, a project group with student IDs 21404312 and 214052104 will create a directory named 21404312-214052104 and will put their files there. Then, they will tar the directory (package the directory) as follows:

tar cvf 21404312-214052104.tar 21404312-214052104

Then they will gzip (compress) the tar file as follows:

gzip 21404312-214052104.tar

In this way they will obtain a file called 21404312-214052104.tar.gz. Then they will upload this file to **Moodle**. For a project done individually, just the ID of the student will be used as a file or directory name.

LATE POLICY FOR THE PROJECT

If the deadline is missed by up to 24 hours, 25 points will be cut from the graded project and that will be the grade of the project. For example, if a late submission is graded and received 80 pts; then the grade of the project will be 80 - 25 = 55. If the deadline is missed between 24 - 48 hours, then 50 pts will be cut. Submission beyond 2 days (48 hours) will not be allowed. Note that even the submission is 1 second later after the deadline, 25 points will be cut. System assigns timestamps to the submissions. We will use those timestamps. Therefore, you are recommended to finish your submission 2-3 hours (at the latest) before the deadline. In fact, it is better if you submit one day before. You can re-submit as many times as you wish by the deadline. Each submission will overwrite the previous one. Please do not send your submission with email. Submission system will be open beyond the deadline up to 48 hours.

CLARIFICATIONS

- Start early, work incrementally.
- Waiting for all children to start:
    - Your library should await *a request for resources* until all child processes are created and have called the rsm_claim function, if avoid == 1.
    - Your library should await *a request for resources* until all child processes are created and have called the rsm_process_started function, if avoid == 0.
    - If you wish you can achieve this at the rsm_process_start(), by not returning until all processes called this function (i.e., set a barrier).
- If you wish you can develop your programs in another OS environment (MacOS, Windows, etc.). But finally, you have to make sure that your programs compile and run properly in Linux. Test them as much as possible. We will test your programs in Linux.
- You are suggested to implement and test deadlock detection first. Then deadlock avoidance.

- You will include a Makefile with your project submission. Make sure it works in your environment (name your files accordingly). We will just type “make” and your programs should compile.
- You can not change the interface file (rsm.h).
- Assuming no deadlock avoidance, we do the following assumption. When a request is made by a process P, either all the requested resources will be allocated to the process P (if they are available), or none of them will be allocated, and request will be blocked (pending). That means no partial allocation will done for a request. s
- When deadlock avoidance is used and when a process requests resources and if it is not safe to allocate resources (they are available but not safe), then the process is not allocated any resources and is blocked. It will stay blocked until resources are available and safe to allocate.
- If you modify rsm.c file, make sure it is compiled when you run "make". To ensure that, first run "make clean", and then "make".
- The submitted version of your library (rsm.c) should not print out anything, except the rsm_print_state() function (it will print the current state). But, while developing and testing your library, it can print messages. You must delete them before submission.
- If avoid == 0, a process will not call rsm_claim().
- If a process is requesting more than its need (in avoidance), the rsm_request() function will return error (-1).
- The library will be aware that a process is there when the process calls the rsm_process_started() function.
- In this project, you will use semaphores only; not mutex/condition variables.
- To make things easier for you in this Project: 1) When a process calls rsm_process_ended(), the application can be terminated after cleanup. That means, you can implement rsm_process_ended() function in such way that, when it is called by a process, it will terminate all the processes (children and main process), will do some cleanup (if needed), and will call exit. That means the whole application terminates when rsm_process_ended() is called. This behaviour for rsm_process_ended() is okay in this project. Hence, you don't need to release resources inside rsm_process_ended() function. 2) If there are deadlocked processes, you don't have to do anything about recovering from deadlock. The processes may stay blocked until they are terminated by some other process or externally.
- You can assume that you will claim once per library initialization. You can assume that in a test program, claim will be done once. Hence you don't need to destroy and init again in the same program/process.
- Since, rsm_process_ended() will terminate everything, destroy() will not have much use anymore. Therefore you can simply implement it similar to ended() function. We will either call ended(), or destroy().
- Extra spaces etc. in the printed output are not problem in this project.
- You may have separate tests / experiments. And all tests and experiment files (.c files) you have used will be uploaded with the submission.

- In this project, you can assume that you will claim once per library initialization. You can assume that in a test program, claim will be done once. Hence
  you don't need to destroy and init again in the same program/process.
- Since, rsm_process_ended() will terminate everything, destroy will not have much use anymore. Therefore you can simply implement it similar to ended() function. We will either call rsm_process_ended(), or rsm_destroy(). rsm_process_ended may be called by a child process. rsm_destroy may be called by the main process.
-

REFERENCES

- POSIX Semaphores. URL: https://man7.org/linux/man-pages/man7/sem_overview.7.html
- POSIX Shared Memory. URL: https://man7.org/linux/man-pages/man7/shm_overview.7.html